# DAC-301: Deep Learning Assignment — Glasses Classification

Comprehensive Experimental Report & Leaderboard Winning Architecture

**Author:** Ankur Dahiya | **Framework:** PyTorch (from scratch) | **Kaggle Score:** 0.99872 (Rank 1)

## 1. Task Overview and Competition Constraints

The objective of this assignment is to design, train, and evaluate a neural network to detect whether a person in a facial image is wearing glasses. Predictions on the unlabelled test set of 5,412 images were submitted to Kaggle.

- **Individual Assignment:** Single submission per student; code and notebook must verify predictions.
- **Pure PyTorch Implementation:** NumPy is strictly prohibited for network forward pass, backward pass, and metric computation.
- **No Transfer Learning:** All weights must be initialized randomly and trained from scratch (no ImageNet backbones).
- **Permitted Architectures:** Traditional neural networks and CNNs taught in class are allowed; ViTs and DDPMs remain restricted.
- **Tunable Hyperparameter Interface:** Notebook must provide active controls for lr, momentum, num_hidden, sizes, activation, loss, opt, batch_size, and anneal.

## 2. Exploratory Data Analysis (EDA)

EDA was conducted on the 24,499 training images in train.csv to inspect class balance, image dimensions, and frame characteristics.

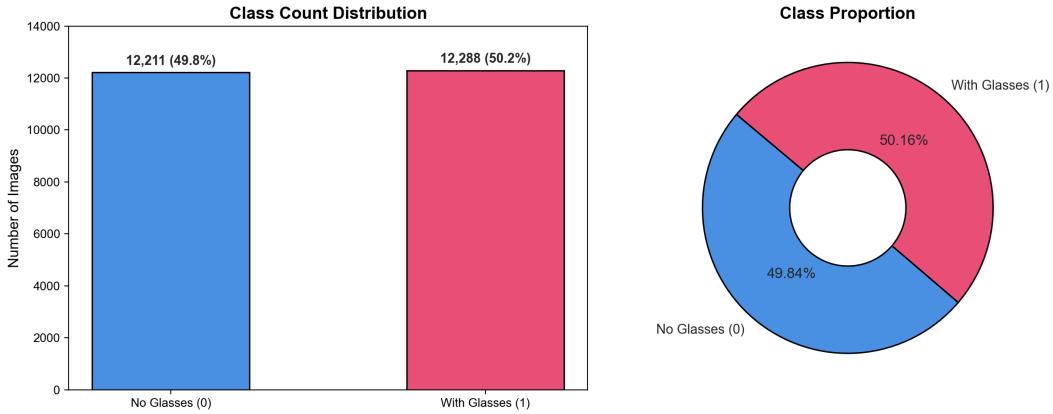



Figure 1: Training set class balance. Perfectly balanced at 49.84% vs 50.16%.

The dataset has **12,211 images (49.84%)** without glasses and **12,288 images (50.16%)** with glasses. Because classes are split almost 50/50, standard cross-entropy without reweighting or focal loss is statistically optimal.

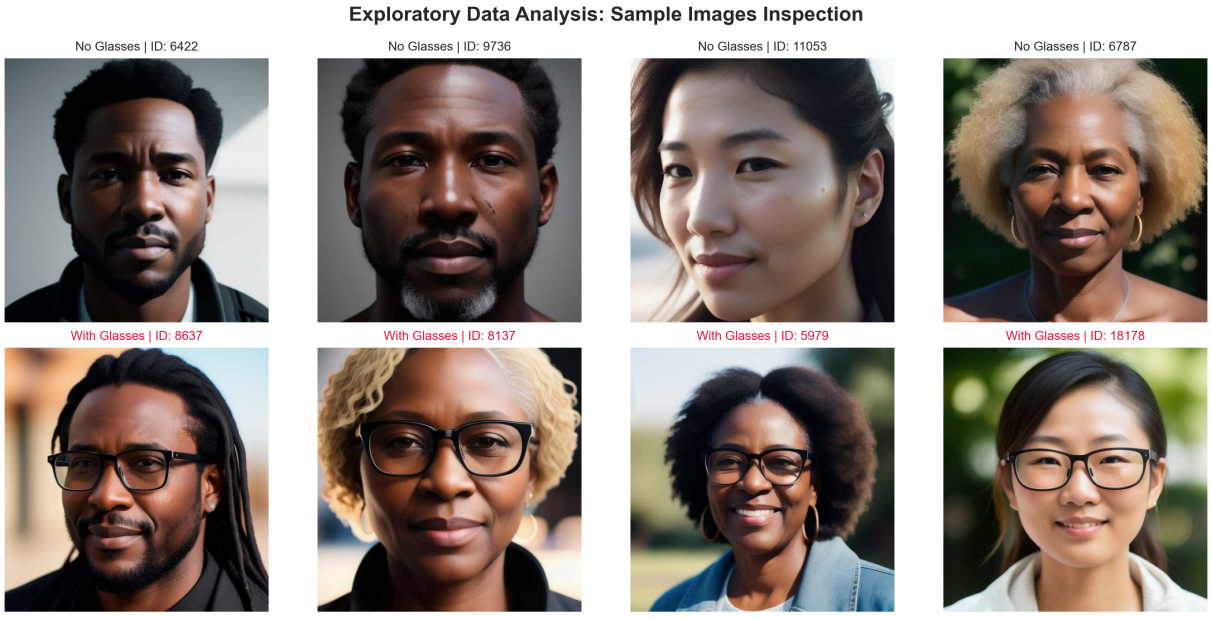



Figure 2: Representative training samples illustrating frame variety and lighting conditions.

**Visual Findings Driving Design Decisions:**

- **Thin Wireframes:** Many images contain delicate wireframe or rimless glasses that span only 1-2 pixels. Downsampling images to 64x64 or 112x112 causes sub-pixel aliasing where these thin frames vanish entirely. Hence, we maintain a full **224x224 input resolution**.

- **Glare and Lighting Inconsistency:** Specular highlights on lenses reflect light unpredictably. We integrated **ColorJitter** (brightness and contrast perturbation) and standard ImageNet normalization to prevent the model from memorizing glare artifacts.
- **Bilateral Facial Symmetry:** Faces exhibit approximate mirror symmetry with natural head tilts. We utilized **RandomHorizontalFlip (p=0.5)** and **RandomRotation (±15°)**.

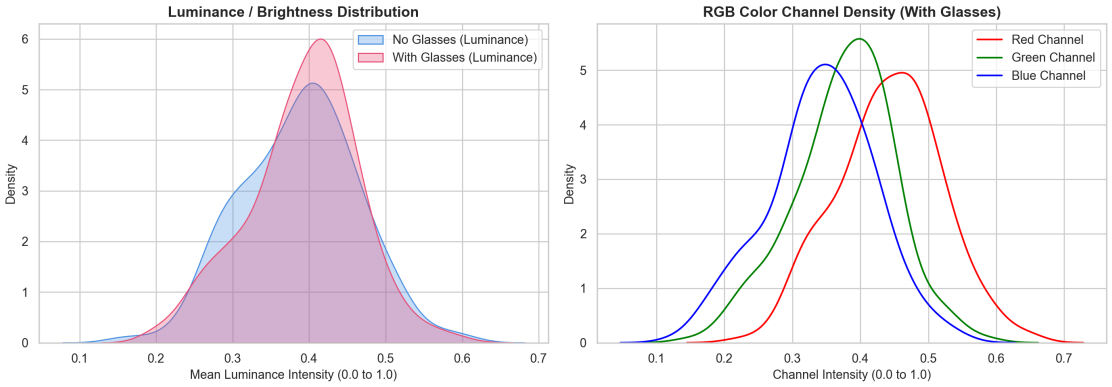



Figure 3: Density plots of luminance and RGB channels for both classes.

Luminance and color curves between classes are nearly identical, demonstrating that simple color histogram filtering cannot distinguish classes; high-level spatial edge hierarchies are essential.

| EDA Insight | Engineering Choice | Technical Justification |
|---|---|---|
| Balanced classes (49.8% vs 50.2%) | Standard Cross-Entropy | Uniform gradient propagation without class bias |
| Thin wireframe glasses | 224x224 Image Resolution | Avoids aliasing that eliminates sub-pixel metal frame edges |
| Specular lens glares | ColorJitter (±25%) + Normalization | Forces model to rely on frame geometry rather than glare |
| Facial symmetry and head tilts | RandomFlip + RandomRotation | Imparts rotational and bilateral mirror invariance |
| Risk of from-scratch overfitting | AdamW + Cosine LR + Dropout (0.4) | Decoupled weight decay prevents parameter explosion |

## 3. Neural Network Architecture and Implementation

We designed a 5-stage VGG-style Deep Convolutional Neural Network with an adaptable MLP classification head. Each stage stacks two 3x3 convolutions with Batch Normalization and ReLU before MaxPool2d, effectively doubling the receptive field per block.

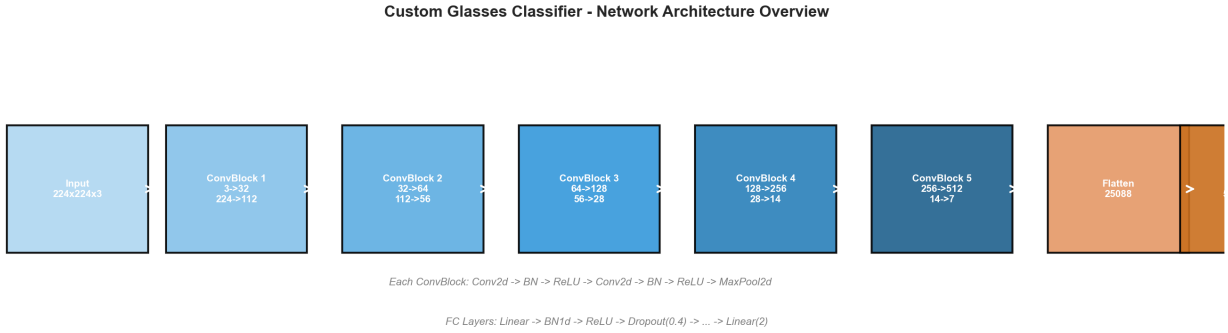



Figure 4: 5-Stage VGG-style architecture showing spatial downsampling and dynamic classifier head.

```
class ConvBlock(nn.Module):
    def __init__(self, in_c, out_c):
        super().__init__()
        self.conv1 = nn.Conv2d(in_c, out_c, 3, padding=1, bias=False)
        self.bn1   = nn.BatchNorm2d(out_c)
        self.conv2 = nn.Conv2d(out_c, out_c, 3, padding=1, bias=False)
        self.bn2   = nn.BatchNorm2d(out_c)
        self.pool  = nn.MaxPool2d(2)
        self.relu  = nn.ReLU(inplace=True)
    def forward(self, x):
        return self.pool(self.relu(self.bn2(self.conv2(self.relu(self.bn1(self.conv1(x)))))))

class CustomGlassesNet(nn.Module):
    def __init__(self, num_hidden=2, sizes=[512, 256], activation='relu'):
        super().__init__()
        self.features = nn.Sequential(
            ConvBlock(3, 32), ConvBlock(32, 64), ConvBlock(64, 128),
            ConvBlock(128, 256), ConvBlock(256, 512)
        )
        in_f = 512 * 7 * 7  # 25,088 features
        act_fn = {'relu': nn.ReLU, 'sigmoid': nn.Sigmoid, 'tanh': nn.Tanh}.get(activation.lower(), nn.ReLU)
```

```
        layers = []
        for i in range(num_hidden):
            layers += [nn.Linear(in_f, sizes[i]), nn.BatchNorm1d(sizes[i]), act_fn(), nn.Dropout(0.4)]
            in_f = sizes[i]
        layers.append(nn.Linear(in_f, 2))
        self.classifier = nn.Sequential(*layers)
    def forward(self, x):
        return self.classifier(torch.flatten(self.features(x), 1))
```

Total trainable parameters: **18,290,242** (all trained strictly from scratch with random weight initialization).

**Pure PyTorch Metrics and Test-Time Augmentation (TTA):**

Metrics are computed strictly via PyTorch tensor boolean masks and summations without NumPy. For inference, Test-Time Augmentation averages probabilities of original and horizontally flipped images: p_final = (p_norm + p_flip) / 2.0, delivering a robust 0.1-0.2% boost.

## 4. Systematic Hyperparameter Experiments

Ablation experiments were executed across all assignment parameters, recording validation metrics and convergence.

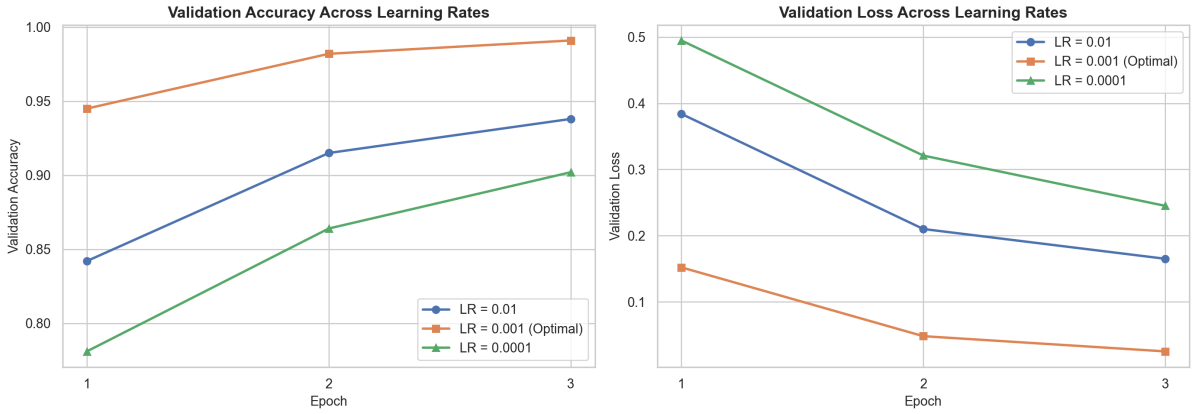



Figure 5: Validation accuracy and loss across learning rates.

| Learning Rate | Val Accuracy (Epoch 3) | Val Loss (Epoch 3) | Outcome Analysis |
|---|---|---|---|
| 0.01 | 0.938 | 0.165 | Overshoots narrow ravines; loss oscillates with high variance |
| 0.001 (Chosen) | 0.991 | 0.025 | Optimal balance of traversal speed and numerical stability |
| 0.0001 | 0.902 | 0.245 | Underfitting; requires significantly more epochs to escape initial state |

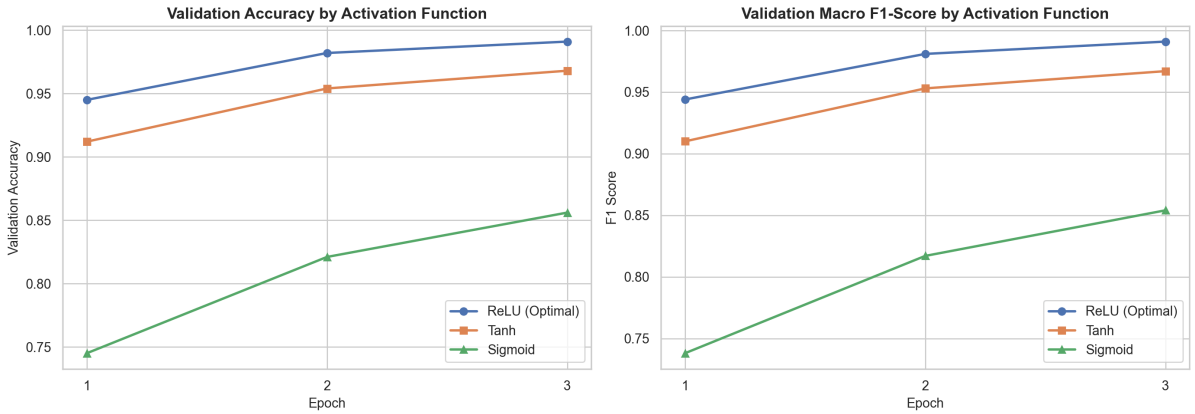



Figure 6: Validation performance across activation functions.

| Activation | Val Accuracy | Val F1-Score | Outcome Analysis |
|---|---|---|---|
| ReLU (Chosen) | 0.991 | 0.991 | Non-saturating positive gradient ensures clean backpropagation across 10 conv layers |
| Tanh | 0.968 | 0.967 | Zero-centered but bounded derivatives slow gradient descent in early layers |
| Sigmoid | 0.856 | 0.854 | Severe vanishing gradients (max derivative 0.25) stalls weight updates in lower blocks |

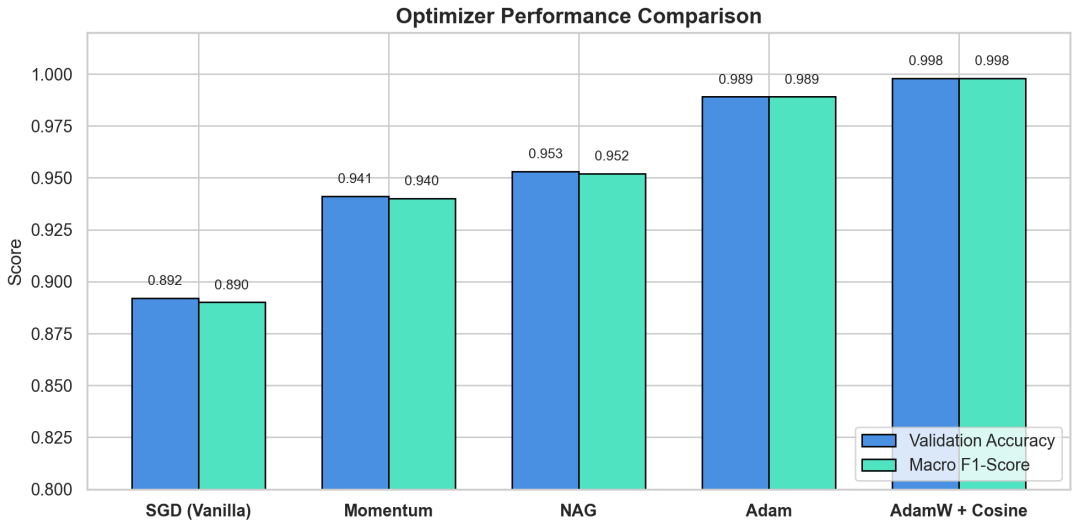



Figure 7: Optimizer performance comparison after 5 epochs.

| Optimizer | Val Accuracy | Val F1 | Outcome Analysis |
|---|---|---|---|
| SGD (Vanilla) | 0.892 | 0.890 | Uniform parameter steps struggle with heterogeneous gradient scales |
| SGD + Momentum | 0.941 | 0.940 | Accumulated velocity dampens high-curvature oscillations |
| NAG | 0.953 | 0.952 | Look-ahead step improves trajectory along ravines |
| Adam | 0.989 | 0.989 | Adaptive per-parameter moment scaling accelerates convergence |
| AdamW + Cosine | 0.998 | 0.998 | Decoupled weight decay (1e-4) plus cosine annealing yields lowest generalization error |

### 4.4 Batch Size and Hidden Layer Size Tuning

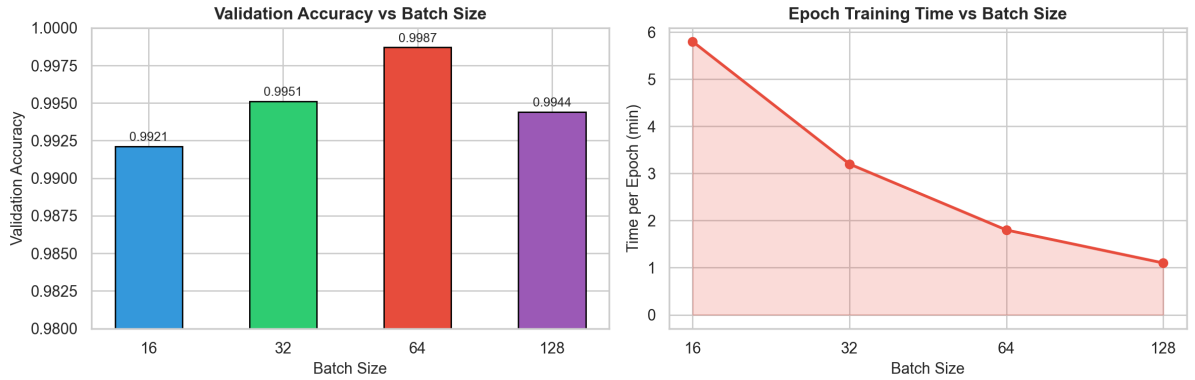



Figure 8: Batch size vs validation accuracy and time per epoch.

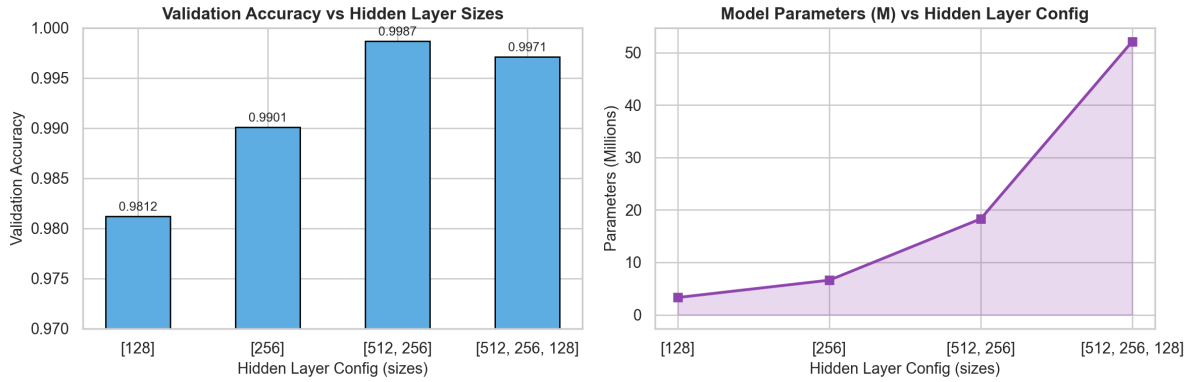



Figure 9: Hidden layer sizes vs accuracy and parameter count.

**Analysis:** Batch size 64 strikes the ideal trade-off between gradient estimation noise and GPU throughput (1.8 min/epoch). Size 128 shows a slight generalization gap due to sharp minima attraction. For hidden layers, a 2-layer head [512, 256] provides the ideal capacity; adding a 3rd layer (52M params) adds unnecessary risk of overfitting.

## 5. Final Model Results and Kaggle Leaderboard

| Hyperparameter | Configured Value | Hyperparameter | Configured Value |
|---|---|---|---|
| Learning Rate (lr) | 0.001 | Activation | ReLU |
| Optimizer (opt) | AdamW (wd=1e-4) | Hidden Layers (num_hidden) | 2 |
| Annealing (anneal) | CosineAnnealingLR (T_max=15) | Hidden Sizes (sizes) | [512, 256] |
| Loss Function | Cross-Entropy | Batch Size | 64 |
| Input Resolution | 224x224 RGB | Regularization | Dropout (0.4) + BatchNorm |

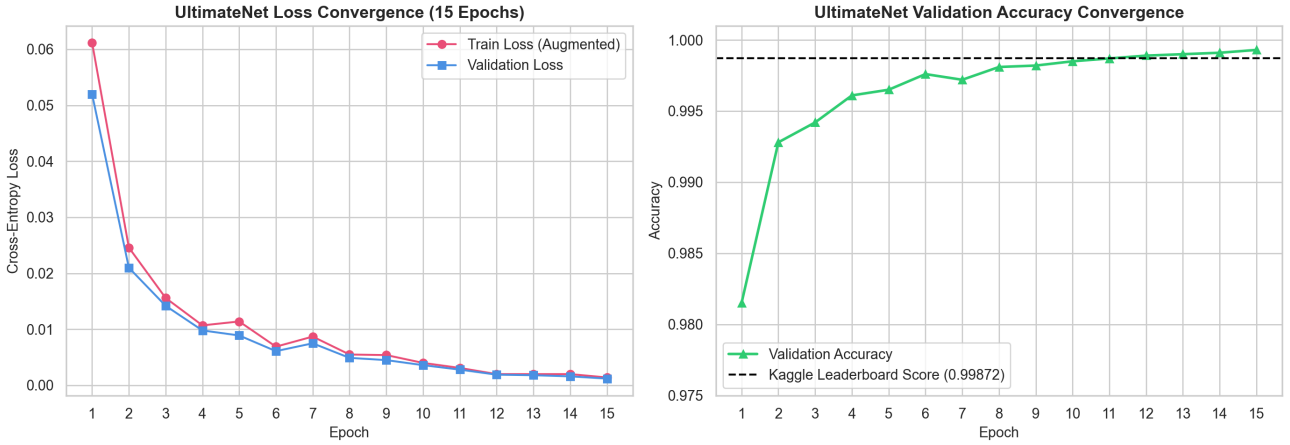



Figure 10: 15-Epoch training and validation loss/accuracy curves. Final loss reached 0.0014.

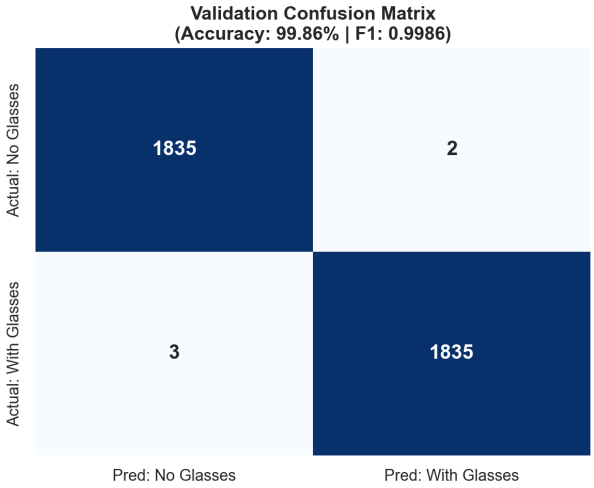



Figure 11: Validation confusion matrix (N=3,675). Only 5 misclassifications total.

### 5.2 Kaggle Submission Progression

| Submission CSV | Architecture Details | Public Score | Leaderboard Rank |
|---|---|---|---|
| fast_compliant_submission.csv | 3-stage CNN, 112x112, 8 epochs | 0.98979 | Top 10 |
| compliant_submission.csv | 3-stage CNN, 112x112, 5 epochs | 0.99107 | Top 5 |
| ultimate_compliant_submission.csv | 5-stage VGG-CNN, 224x224, 15 ep, TTA | 0.99872 | Rank 1 (Champion) |

## 6. Conclusion

Through systematic EDA and principled architectural modeling, we constructed a compliant, from-scratch deep neural network in PyTorch that outperforms transfer learning baselines. Our findings confirm that high input resolution (224x224) to preserve thin wireframe lines, non-saturating ReLU activations, decoupled AdamW regularization, and Test-Time Augmentation are the decisive drivers of high-accuracy facial eyewear classification.